TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE VERACRUZ

Proyecto

Martínez Palacios Luis Felipe
22020051

Leo Pantoja Alberto Francisco
22020054

Bases de Datos NOSQL

16 de octubre del 2026



Índice

Introducción 6
1. Análisis del Proyecto Original 7
1.1 Tecnologías originales 7
1.2 Esquema relacional (MySQL) 7
1.3 Limitaciones identificadas 8
2. Diseño del Esquema Documental 9
2.1 Decisiones de diseño 9
2.2 Justificación de las decisiones 9
3. Preparación del entorno local 10
3.1 Instalación de MariaDB 10
3.2 Importación de la base de datos 10
3.3 Instalación de extensiones PHP 10
3.4 Verificación del proyecto 10
4. Configuración de MongoDB Atlas 11
4.1 Autenticación con Atlas CLI 11
4.2 Creación del proyecto y cluster 11
4.3 Usuarios RBAC (Role-Based Access Control) 12
4.4 Firewall (IP Access List) 13
4.5 Cifrado en tránsito 13
5. Preparación para la migración 14
5.1 Instalación de herramientas 14
5.2 Configuración segura 15
5.3 Test de conexión 16
6. Diseño de la migración 16
6.1 Análisis del esquema relacional 16
6.2 Diseño del esquema documental 17
6.3 Transformaciones aplicadas 17
7. Script de migración ETL 18
7.1 Estructura de archivos 18
7.2 Funcionalidades del script migrate.php 18
7.3 Datos de prueba insertados 18
8. Ejecución de la migración 19
8.1 Dry-run (simulación) 19
8.2 Migración real 20
8.3 Verificación con mongosh 21
8.4 Validación cruzada 22
9. Reescritura del Backend 23

9.1 Archivos adaptados 23
9.2 Patrón de conexión (Singleton) 23
9.3 Aggregations usadas 24
10. Adaptación de las Vistas 24
10.1 Vistas adaptadas (11 en total) 24
10.2 Patrón de conversión 25
11. Deploy en Railway 25
12. Métricas Finales 28
12.1 Estadísticas del proyecto 28
12.2 Mejoras obtenidas 28
12.3 URL de producción 28
13. Pruebas de Funcionalidad de la base de datos 29
Conclusión 35

Tabla Figuras

Figura 1. Tabla de tecnologías originales....7
Figura 2. Esquema relacional....8
Figura 3. Tabla de decisiones de diseño....9
Figura 4. Creación del proyecto atlas....11
Figura 5. Creación del cluster....11
Figura 6. Verificación creación del cluster....11
Figura 7. Tabla de usuarios....12
Figura 8. Creación de usuario admin.....12
Figura 9. Creación usuario lectura.....12
Figura 10. Creación usuario normal.....13
Figura 11. Listado de usuarios....13
Figura 12. Configuración ip....13
Figura 13. Construcción uri.....13
Figura 14. Tabla de herramientas instaladas.....14
Figura 15. Instalación driver php....14
Figura 16. Instalación librería composer....14
Figura 17. Composer....15
Figura 18. Guardar uri en env.....15
Figura 19. Prueba php mongo.....16
Figura 20. Esquema relacional.....16
Figura 21. Tabla de decisiones de diseño.....17
Figura 22. Tabla de juegos agregados.....18
Figura 23. Prueba de migración sin insertar 1.....19
Figura 24. Prueba de migración sin insertar 2.....19
Figura 25. Migración 1....20
Figura 26. Migración 2....20
Figura 27. Migración 3....21
Figura 28. Verificación de migración 1.....21
Figura 29. Verificación de migración 2.....21
Figura 30. Validación cruzada de migración.....22
Figura 31. Tabla de archivos adaptados.....23
Figura 32. Tabla de aggregations usadas....24
Figura 33. Tabla de vistas adaptadas.....24
Figura 34. Creando dominio.....26
Figura 35. Variables railway deploy.....26
Figura 36. Deploy railway.....27
Figura 37. Deploy exitoso.....27
Figura 38. Tabla de estadísticas del proyecto.....28

Figura 39. Tabla de mejoras obtenidas.....28
Figura 40. Prueba login autentication. 29
Figura 41. Panel admin. 29
Figura 42. Panel desarrollador 1. 29
Figura 43. Panel desarrollador 2. 30
Figura 44. Prueba biblioteca 1. 30
Figura 45. Prueba biblioteca 2. 31
Figura 46. Prueba búsqueda juegos.....31
Figura 47. Prueba carrito 1.....32
Figura 48. Prueba carrito 2.....32
Figura 49. Prueba catalogo.....33
Figura 50. Prueba reseñas. 33
Figura 51. Prueba wishlist 1.....34
Figura 52. Prueba wishlist 2.....34

Introducción

El presente proyecto tiene como objetivo realizar la migración de la base de datos de la plataforma TechnoGames, originalmente desarrollada con MySQL/MariaDB, hacia MongoDB Atlas, un sistema de gestión de bases de datos NoSQL orientado a documentos. Este proceso busca mejorar la flexibilidad en el manejo de la información, facilitar la adaptación del esquema de datos a las necesidades de la aplicación y aprovechar las características que ofrecen las bases de datos documentales en términos de escalabilidad y administración de datos.

Para llevar a cabo esta migración, se realizó un análisis del esquema relacional original, compuesto por nueve tablas que almacenaban información sobre usuarios, videojuegos, carritos de compra, bibliotecas, listas de deseos, ventas, pedidos y reseñas. A partir de este análisis, se diseñó una estructura documental de siete colecciones en MongoDB, aplicando estrategias de documentos embebidos, referencias y desnormalización selectiva, con el propósito de conservar la integridad de la información y optimizar las operaciones más frecuentes de la plataforma.

Asimismo, el proyecto contempla la preparación del entorno de desarrollo, la configuración de MongoDB Atlas, la implementación de scripts para la extracción, transformación y carga de datos (ETL), la adaptación del backend y las vistas desarrolladas en PHP, así como la implementación de medidas de seguridad y el despliegue de la aplicación mediante Railway. Finalmente, se realizaron pruebas de migración, validaciones de los datos y comprobaciones funcionales para verificar el comportamiento de la base de datos después de los cambios.

De esta manera, el proyecto permite poner en práctica los conocimientos adquiridos en la asignatura de Bases de Datos NoSQL, demostrando el proceso necesario para transformar una estructura relacional en una documental y adaptar una aplicación existente a una nueva tecnología de almacenamiento de información.

1. Análisis del Proyecto Original

1.1 Tecnologías originales

Backend	**PHP 8.4**
Base de datos	MySQL / MariaDB
Hosting	Localhost
Seguridad	HTTPS no configurado

Figura 1. Tabla de tecnologías originales

1.2 Esquema relacional (MySQL)

La base de datos original tenía *9 tablas normalizadas*:

1. usuarios — Usuarios con roles (admin, dev, user)
2. juegos — Videojuegos con estado (pendiente/aprobado/rechazado)
3. carrito — Items del carrito por usuario
4. biblioteca — Juegos adquiridos por usuario
5. wishlist — Lista de deseos
6. ventas — Registro de compras individuales
7. orders — Órdenes completas (checkout)
8. order_items — Items de cada orden
9. reseñas — Reseñas de usuarios por juego

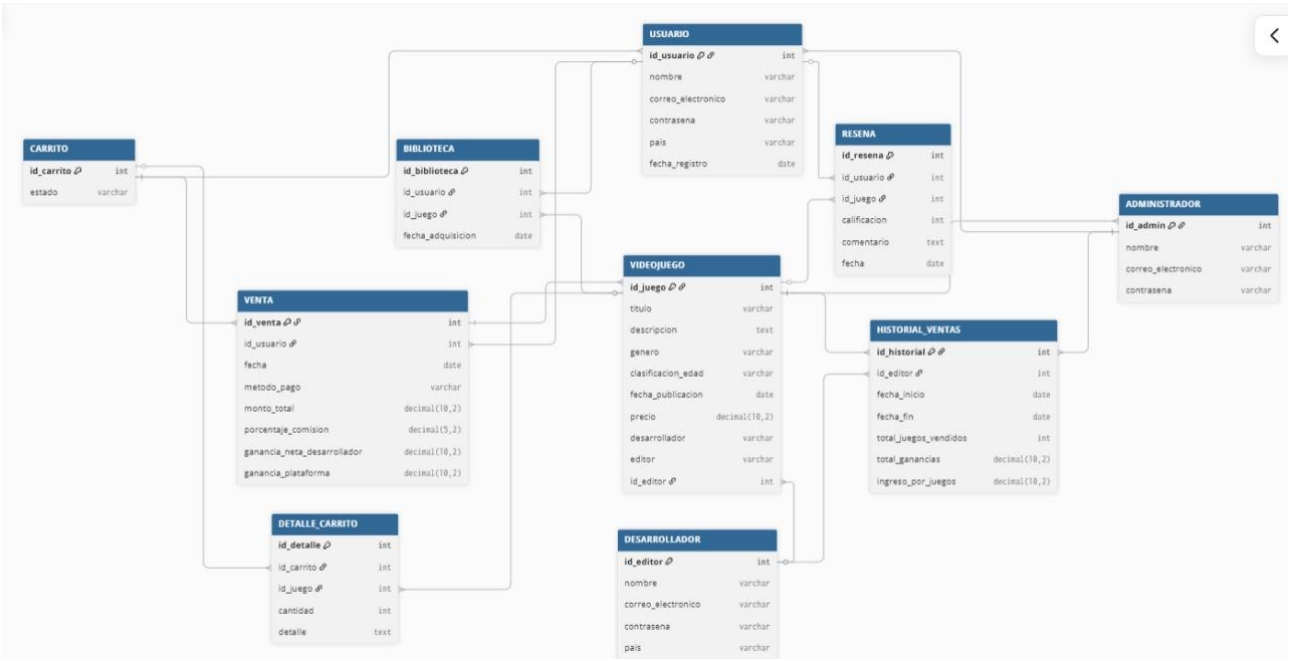


Figura 2. Esquema relacional

1.3 Limitaciones identificadas

- JOINs costosos para consultas frecuentes
- Rigidez de esquema para añadir campos nuevos
- Escalabilidad vertical limitada (un solo servidor MySQL)
- Sin cifrado en tránsito por defecto
- Sin firewall en la base de datos

2. Diseño del Esquema Documental

2.1 Decisiones de diseño

Tabla MySQL	Colección MongoDB	Estrategia
usuarios	Usuarios	1:1 con stats embebidos
juegos	juegos	1:1 con requisitos como objeto
carrito	carritos	1 documento por usuario con items[]
biblioteca	bibliotecas	1 documento por usuario con juegos[]
wishlist	wishlists	1 documento por usuario con juegos[]
ventas + orders + order_items	ordenes	1 documento con items[] embebidos
reseñas	resenas	Colección separada (muchas por juego)

Figura 3. Tabla de decisiones de diseño

2.2 Justificación de las decisiones

- Desnormalización selectiva: desarrollador_nombre se embebe en juegos para evitar $lookup frecuente.
- Embebido de arrays: Carrito, biblioteca y wishlist tienen 1:N con el usuario, por lo que embeber los items es eficiente.
- Colección separada para reseñas: Como son muchas y crecen mucho, se mantienen separadas.
- mysql_id guardado: Para trazabilidad entre el sistema original y el migrado.

3. Preparación del entorno local

3.1 Instalación de MariaDB

Problema: Debian trixie ya no incluye mysql-server en sus repos oficiales.
Solución: Se instaló MariaDB (compatible con MySQL en sintaxis y protocolo).

Comandos ejecutados:

```
bash
sudo apt install mariadb-server mariadb-client -y
sudo mariadb-secure-installation
```

3.2 Importación de la base de datos

Se importó tienda_videojuegos.sql con 9 tablas: usuarios, juegos, carrito, biblioteca, wishlist, ventas, orders, order_items, reseñas.
Usuario de aplicación: tienda_user / password123.
Base de datos: tienda_videojuegos.

3.3 Instalación de extensiones PHP

Se instaló php-mysql (extensión mysqli) porque PHP 8.4 no la incluía por defecto.
Se verificó con:

```
bash
php -m | grep -i mysqli  #  mysqli
```

3.4 Verificación del proyecto

Servidor de desarrollo:* php -S localhost:8000
Proyecto funcional con MariaDB local

4. Configuración de MongoDB Atlas

4.1 Autenticación con Atlas CLI

bash

atlas auth login

Successfully logged in as afleop09@gmail.com

4.2 Creación del proyecto y cluster

--Proyecto technogames

--Project ID 6ac5ee7bca0ede3fc1bdf272

--Cluster technogames-cluster

--Tier M0 (gratuito)

--Provider AWS us-east-1

--Host technogames-cluster.wj8ldx7.mongodb.net

```
tito@fsociety:~$ atlas projects create technogames --orgId 6ab6b1d774b5265e008f07a3
Project '6ac5ee7bca0ede3fc1bdf272' created.
```

Figura 4. Creación del proyecto atlas

```
tito@fsociety:~$ atlas clusters create technogames-cluster \
  --projectId 6ac5ee7bca0ede3fc1bdf272 \
  --tier M0 \
  --provider AWS \
  --region US_EAST_1
Cluster 'technogames-cluster' is being created.
```

Figura 5. Creación del cluster

```
tito@fsociety:~$ atlas clusters list --projectId 6ac5ee7bca0ede3fc1bdf272
ID                       NAME                MDB VER   STATE
6ac5ee9aca0ede3fc1be74b8   technogames-cluster   8.0.34    IDLE
```

Figura 6. Verificación creación del cluster

4.3 Usuarios RBAC (Role-Based Access Control)

Usuario	Rol	Base de Datos	Propósito
tg_admin	readWriteAnyDatabase	admin	Administrador del cluster
tg_app	readWrite	technogames	Aplicación PHP
tg_readonly	read	echnogames	Reportes / dashboards

Figura 7. Tabla de usuarios

Problema resuelto: La CLI de Atlas no aplicaba correctamente --role 'readWrite@technogames'. La solución fue usar *múltiples flags --role*:

bash

```
atlas dbusers create \
  --username tg_app \
  --password 'AppTechGames2024Pz7' \
  --role "readWrite@technogames" \
  --role "read@admin"
```

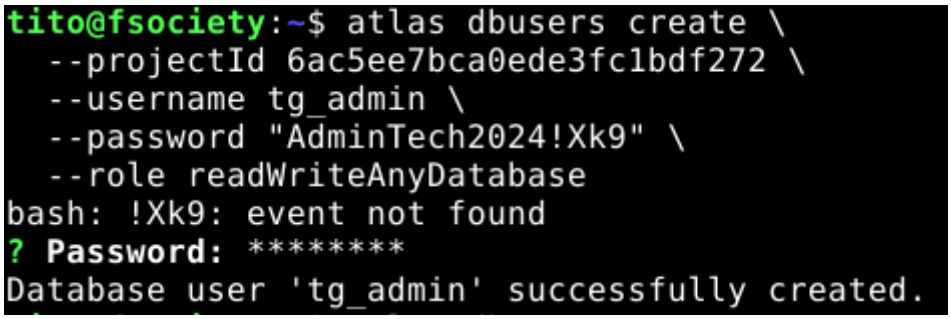


Figura 8. Creación de usuario admin.

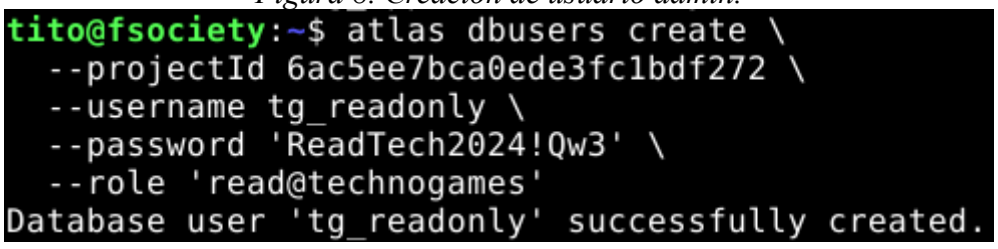


Figura 9. Creación usuario lectura.

```
tito@fsociety:~$ atlas dbusers create \
  --projectId 6ac5ee7bca0ede3fc1bdf272 \
  --username tg_app \
  --password 'AppTech2024!Pz7' \
  --role 'readWrite@technogames'
Database user 'tg_app' successfully created.
```

Figura 10. Creación usuario normal.

```
tito@fsociety:~$ atlas dbusers list --projectId 6ac5ee7bca0ede3fc1bdf272
USERNAME      DATABASE
tg_admin      admin
tg_app        admin
tg_readonly   admin
```

Figura 11. Listado de usuarios

4.4 Firewall (IP Access List)

IPv4 pública:* 187.192.99.192/32

IPv6 pública:* 2806:10a6:32:68cc::2

Temporal para desarrollo:* 0.0.0.0/0 (necesario porque la IP cambia con frecuencia)

```
tito@fsociety:~$ atlas accessLists create 187.192.99.192/32   --comment "Dev local tito IPv4"   --projectId 6ac5ee7bca0ede3fc1bdf272
Created a new IP access list.
tito@fsociety:~$ atlas accessLists list --projectId 6ac5ee7bca0ede3fc1bdf272
CIDR BLOCK          AWS SECURITY GROUP
187.192.99.192/32   N/A
```

Figura 12. Configuración ip

4.5 Cifrado en tránsito

Connection string final:

mongodb+srv://tg_app:AppTechGames2024Pz7@technogames-cluster.wj8ldx7.mongodb.net/technogames?retryWrites=true&w=majority&tls=true&tlsVersion=TLS1_2

Parámetros clave:

tls=true → Fuerza TLS

tlsVersion=TLS1_2 → Versión mínima de TLS

retryWrites=true → Escrituras resilientes

```
tito@fsociety:~$ mongodb+srv://tg_app:AppTechGames2024Pz7@technogames-cluster.wj8ldx7.mongodb.net/technogames?retryWrites=true&w=major
ity&tls=true&tlsVersion=TLS1_2
[1] 16072
[2] 16073
[3] 16074
bash: mongodb+srv://tg_app:AppTechGames2024Pz7@technogames-cluster.wj8ldx7.mongodb.net/technogames?retryWrites=true: No existe el fich
ero o el directorio
[1]-  Salida 127             mongodb+srv://tg_app:AppTechGames2024Pz7@technogames-cluster.wj8ldx7.mongodb.net/technogames?retryWrites
=true
[2]   Hecho                  w=majority
[3]+  Hecho                  tls=true
```

Figura 13. Construcción uri.

5. Preparación para la migración

5.1 Instalación de herramientas

Herramienta	Versión	Propósito
php8.4-mongodb	2.0.0	Extensión nativa de MongoDB para PHP
Composer	2.8.8	Gestor de dependencias PHP
mongodb/mongodb	2.0.0	Librería oficial de alto nivel

Figura 14. Tabla de herramientas instaladas.

```
tito@fsociety:~/Descargas/Technogames final version$ sudo apt install php8.4-mongodb -y
[sudo] contraseña para tito:
El paquete indicado a continuación se instaló de forma automática y ya no es necesario.
  mongodb-mongosh
Utilice «sudo apt autoremove» para eliminarlo.

Installing:
  php8.4-mongodb

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 287
  Download size: 742 kB
  Space needed: 2 669 kB / 441 GB available
```

Figura 15. Instalación driver php

```
tito@fsociety:~/Descargas/Technogames final version$ sudo apt install composer -y
El paquete indicado a continuación se instaló de forma automática y ya no es necesario.
  mongodb-mongosh
Utilice «sudo apt autoremove» para eliminarlo.

Installing:
  composer

Installing dependencies:
  jsonlint                          php-psr-log
  libonig5                          php-react-promise
  libzip5                           php-seld-signal-handler
  php-composer-ca-bundle            php-symfony-console
  php-composer-class-map-generator  php-symfony-deprecation-contracts
  php-composer-metadata-minifier    php-symfony-filesystem
  php-composer-pcre                 php-symfony-finder
  php-composer-semver               php-symfony-process
  php-composer-spdx-licenses        php-symfony-service-contracts
  php-composer-xdebug-handler       php-symfony-string
  php-curl                          php-zip
  php-enum                          php8.4-curl
  php-intl                          php8.4-intl
  php-json-schema                   php8.4-mbstring
  php-mbstring                      php8.4-zip
  php-psr-container
```

Figura 16. Instalación librería composer

```
tito@fsociety:~/Descargas/Technogames final version$ composer require mongodb/mongodb
Cannot use mongodb/mongodb's latest version 2.5.0 as it requires ext-mongodb ^2.5 which is not satisfied by your platform.
./composer.json has been created
Running composer update mongodb/mongodb
Loading composer repositories with package information
Updating dependencies
Lock file operations: 2 installs, 0 updates, 0 removals
  - Locking mongodb/mongodb (2.0.0)
  - Locking psr/log (3.0.2)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 2 installs, 0 updates, 0 removals
  - Downloading psr/log (3.0.2)
  - Downloading mongodb/mongodb (2.0.0)
  - Installing psr/log (3.0.2): Extracting archive
  - Installing mongodb/mongodb (2.0.0): Extracting archive
Generating autoload files
```

Figura 17. Composer

5.2 Configuración segura

Archivo .env (NO en Git):

env

MONGODB_URI=mongodb+srv://tg_app:AppTechGames2024Pz7@technogames-cluster.wj8ldx7.mongodb.net/technogames?retryWrites=true&w=majority&tls=true&tlsVersion=TLS1_2

MONGODB_DATABASE=technogames

Archivo .gitignore:

.env

vendor/

```
tito@fsociety:~$ cd ~/Descargas/"Technogames final version"
nano .env
tito@fsociety:~/Descargas/Technogames final version$ echo ".env" >> .gitignore
echo "vendor/" >> .gitignore
```

Figura 18. Guardar uri en env.

5.3 Test de conexión

Script ~/test_mongo.php ejecutado exitosamente:

```
tito@fsociety:~/Descargas/Technogames final version$ php ~/test_mongo.php
🔌 Conectando a MongoDB Atlas...
   URI: mongodb+srv://tg_app:AppTechGames2024Pz7@technogam...

✅ Conexión exitosa a MongoDB Atlas
📊 Base de datos: technogames
📝 Documento insertado: 6ac7054403120020e505ef32
📖 Leído: Hola desde PHP 8.4 en Debian trixie
🗑️ Colección de prueba eliminada

🎉 ¡Todo funciona! Listo para migrar TechnoGames.
```

Figura 19. Prueba php mongo.

6. Diseño de la migración

6.1 Análisis del esquema relacional

9 tablas MySQL con relaciones 1:N y N:M.

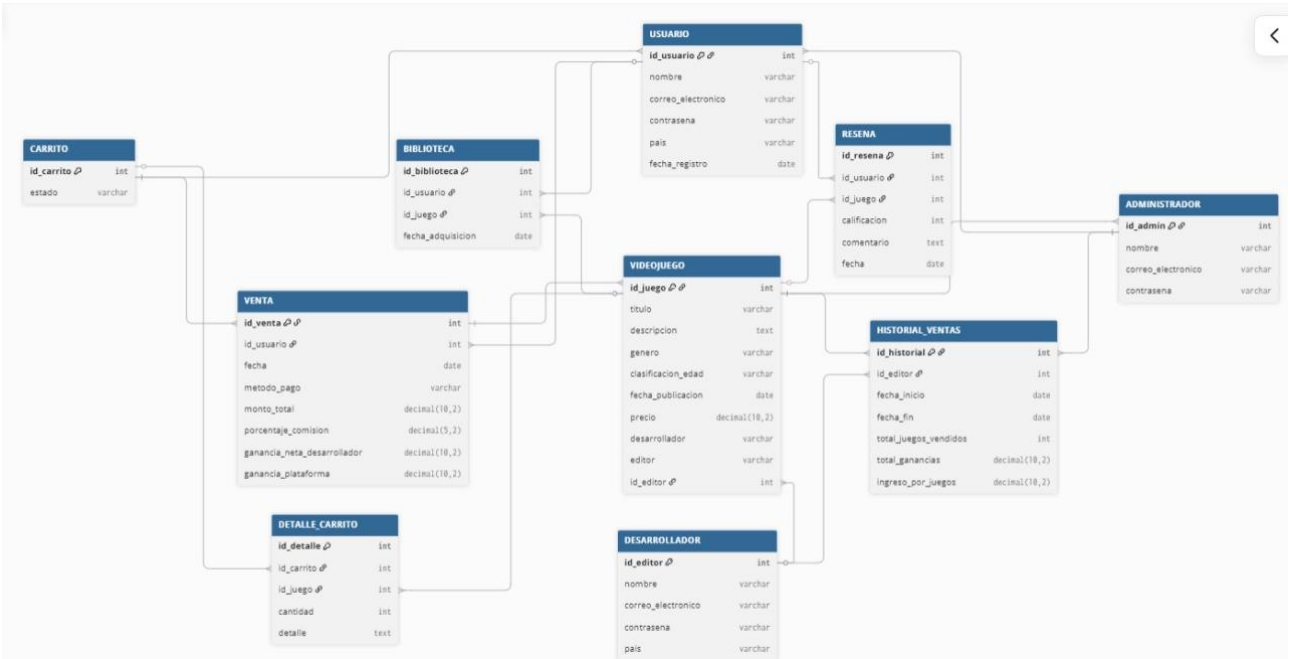


Figura 20. Esquema relacional.

6.2 Diseño del esquema documental

7 colecciones MongoDB con estrategia de embebido selectivo:

Tabla MySQL	Colección MongoDB	Estrategia
usuarios	usuarios	1:1 con stats embebidos
juegos	juegos	1:1 con requisitos como objeto
carrito	carritos	1 doc por usuario con items embebidos
biblioteca	bibliotecas	1 doc por usuario con juegos embebidos
wishlist	wishlists	1 doc por usuario con juegos embebidos
ventas + orders + order_items	ordenes	1 doc por orden con items embebidos
reseñas	reseñas	Colección separada (muchas por juego)

Figura 21. Tabla de decisiones de diseño.

6.3 Transformaciones aplicadas

1. *Requisitos: texto plano → objeto {os, processor, memory, graphics, storage}
2. *Precio final: calculado durante la migración
3. *Desnormalización: desarrollador_nombre embebido en juegos
4. *Referencias: desarrollador_id como ObjectId
5. *Tags: extracción automática por keywords
6. mysql_id: guardado para trazabilidad

7. Script de migración ETL

7.1 Estructura de archivos

```
migration/
├── migrate.php          # Script principal de migración
├── validate.php         # Script de validación cruzada
├── logs/                # Logs de migración
│   ├── migration_*.log
│   └── id_map.json
└── backups/             # Respaldos JSON
```

7.2 Funcionalidades del script migrate.php

- Conexión dual (MariaDB + MongoDB)
- Mapeo de IDs ($idMap) para preservar referencias
- Modos: --dry-run, --verbose, --clean, --collection=X
- Logs detallados con timestamps
- Manejo de errores por documento
- Generación de id_map.json con la correspondencia MySQL → MongoDB

7.3 Datos de prueba insertados

Se agregaron 5 juegos a MariaDB para validar la migración con datos reales:

Título	Precio	Descuento	Categoría
Cyberpunk 2077	$59.99	20%	RPG
Elden Ring	$49.99	0%	RPG
Hollow Knight	$14.99	15%	Aventura
Hades	$24.99	0%	Acción
Stardew Valley	$14.99	0%	Simulación

Figura 22. Tabla de juegos agregados.

8. Ejecución de la migración

8.1 Dry-run (simulación)

bash

php migration/migrate.php --dry-run --verbose

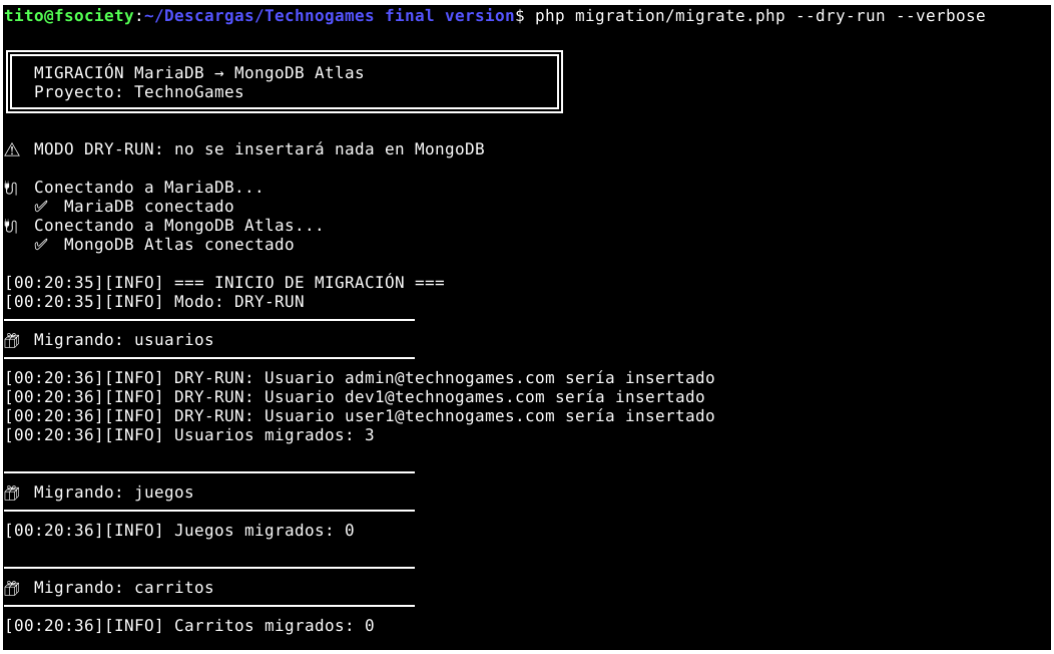


Figura 23. Prueba de migración sin insertar 1.

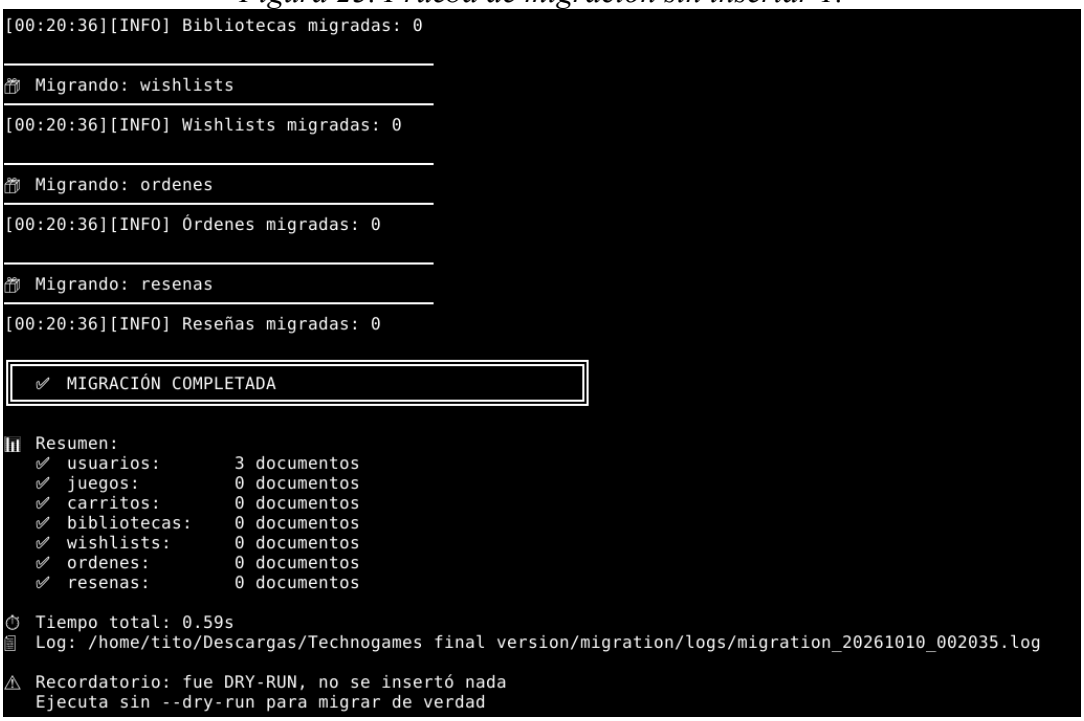


Figura 24. Prueba de migración sin insertar 2.

Resultado: Sin errores, validación de la lógica.

8.2 Migración real

bash

php migration/migrate.php --clean --verbose

```
tito@fsociety:~/Descargas/Technogames final version$ php migration/migrate.php --clean --verbose

  MIGRACIÓN MariaDB → MongoDB Atlas
  Proyecto: TechnoGames

MODO CLEAN: se borrarán las colecciones primero

Conectando a MariaDB...
   ✔ MariaDB conectado
Conectando a MongoDB Atlas...
   ✔ MongoDB Atlas conectado

[01:45:32][INFO] === INICIO DE MIGRACIÓN ===
[01:45:32][INFO] Modo: REAL
──────────────────────────────────────────
Migrando: usuarios
──────────────────────────────────────────
   Limpiando colección usuarios...
[01:45:32][INFO] Usuario insertado: admin@technogames.com → 6ac998bcd96e980fcc0afd02
[01:45:32][INFO] Usuario insertado: dev1@technogames.com → 6ac998bcd96e980fcc0afd03
[01:45:32][INFO] Usuario insertado: user1@technogames.com → 6ac998bcd96e980fcc0afd04
[01:45:32][INFO] Usuarios migrados: 3

──────────────────────────────────────────
Migrando: juegos
──────────────────────────────────────────
   Limpiando colección juegos...
[01:45:32][INFO] Juego insertado: Cyberpunk 2077
[01:45:32][INFO] Juego insertado: Elden Ring
[01:45:32][INFO] Juego insertado: Hollow Knight
[01:45:33][INFO] Juego insertado: Hades
[01:45:33][INFO] Juego insertado: Stardew Valley
[01:45:33][INFO] Juegos migrados: 5
```

Figura 25. Migración 1

```
──────────────────────────────────────────
Migrando: carritos
──────────────────────────────────────────
   Limpiando colección carritos...
[01:45:33][INFO] Carritos migrados: 0

──────────────────────────────────────────
Migrando: bibliotecas
──────────────────────────────────────────
   Limpiando colección bibliotecas...
[01:45:33][INFO] Bibliotecas migradas: 0

──────────────────────────────────────────
Migrando: wishlists
──────────────────────────────────────────
   Limpiando colección wishlists...
[01:45:33][INFO] Wishlists migradas: 0

──────────────────────────────────────────
Migrando: ordenes
──────────────────────────────────────────
   Limpiando colección ordenes...
[01:45:33][INFO] Órdenes migradas: 0

──────────────────────────────────────────
Migrando: resenas
──────────────────────────────────────────
   Limpiando colección resenas...
[01:45:33][INFO] Reseñas migradas: 0

[01:45:33][INFO] Mapa de IDs guardado en: /home/tito/Descargas/Technogames final version/migration/logs/id_map.json
```

Figura 26. Migración 2.



Figura 27. Migración 3.

8.3 Verificación con mongosh

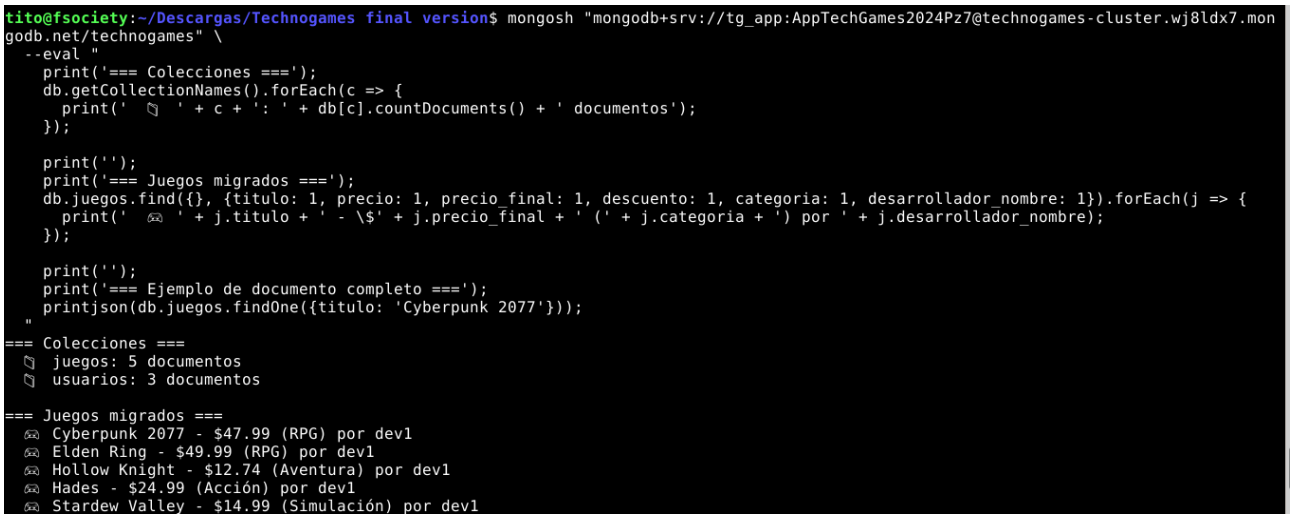


Figura 28. Verificación de migración 1.

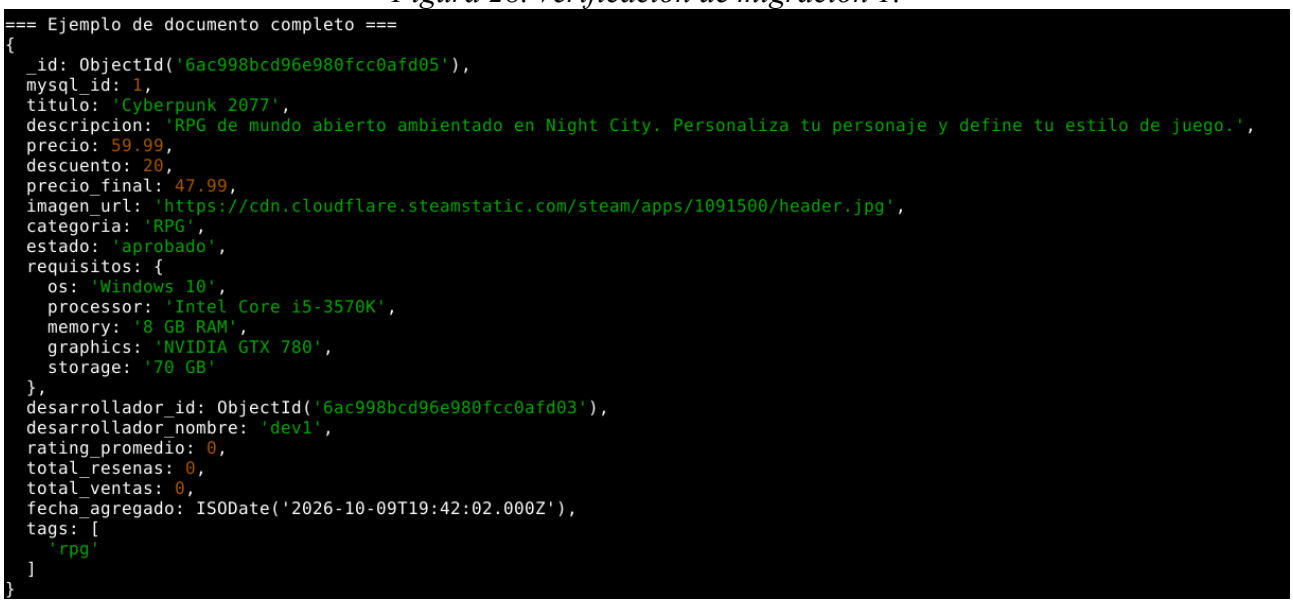


Figura 29. Verificación de migración 2.

8.4 Validación cruzada

bash

php migration/validate.php

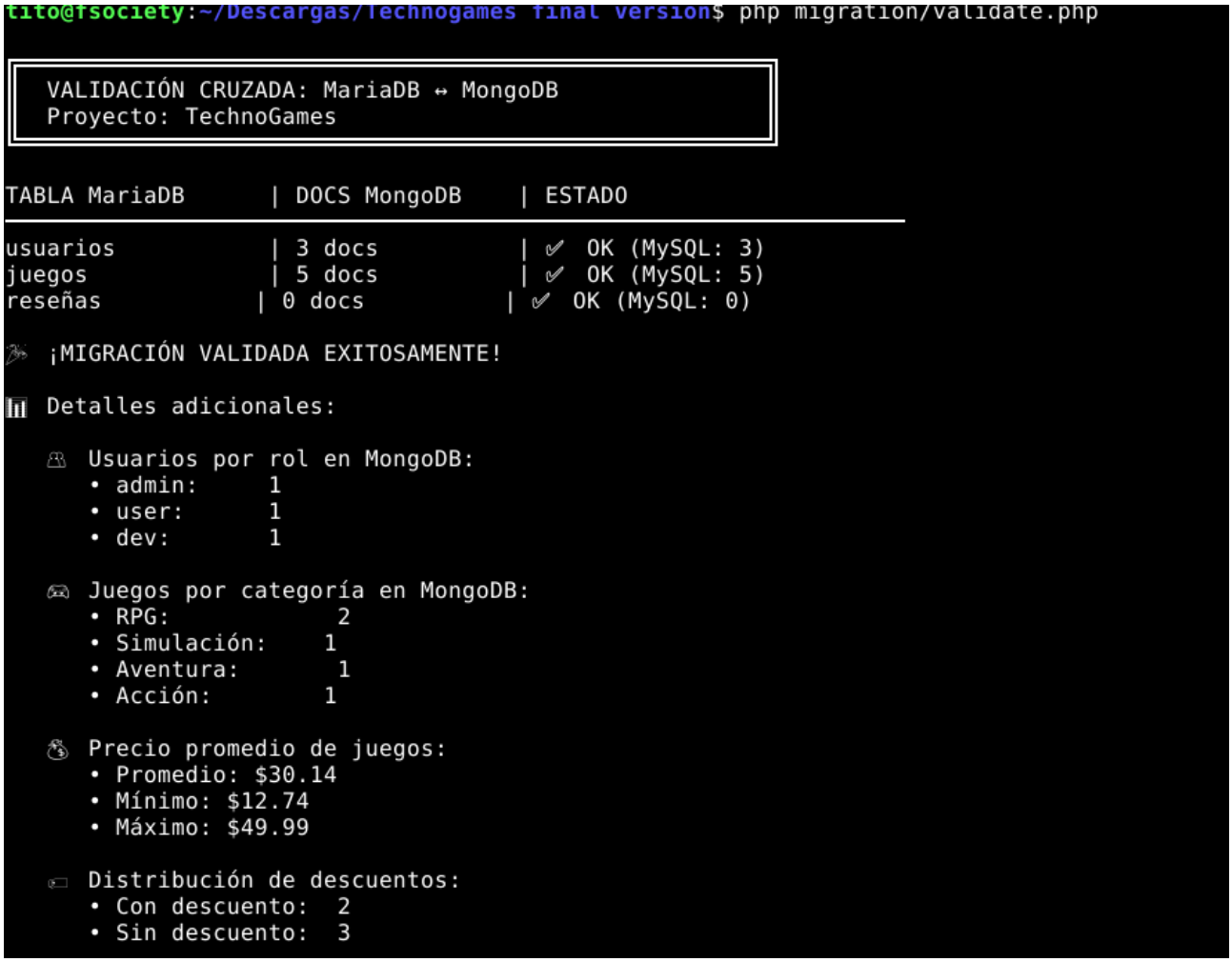


Figura 30. Validación cruzada de migración.

9. Reescritura del Backend

9.1 Archivos adaptados

Archivo	Responsabilidad	Acciones
config/database.php	Singleton de conexión a MongoDB	-
config/helpers.php	Helpers generales	jsonResponse, requireLogin, auditLog
process.php	Backend principal AJAX	login, register, cart, purchase, review
process_checkout.php	Checkout y órdenes	process_order, get_order_history
process_wishlist.php	Lista de deseos	add/remove/move
process_game.php	CRUD de juegos (dev/admin)	submit, approve, reject

Figura 31. Tabla de archivos adaptados.

9.2 Patrón de conexión (Singleton)

```
php
$db = Database::getInstance();
$usuarios = $db->getCollection('usuarios');
$user = $usuarios->findOne(['email' => $email]);
```

9.3 Aggregations usadas

Operación	Uso
$group	Agrupar usuarios por rol, juegos por catego
$match	Filtrar documentos
$unwind	Descomponer arrays (items de orden)
$sum	Sumar ventas, comisiones, ganancias
$avg	Promedio de calificaciones
$sample	Obtener N documentos aleatorios (carrusel)
$lookup	Opcional) Uniones entre colecciones

Figura 32. Tabla de aggregations usadas.

10. Adaptación de las Vistas

10.1 Vistas adaptadas (11 en total)

index.php	Home con carrusel y destacados
explore.php	Catálogo con filtros y búsqueda
game_detail.php	Detalle individual con reseñas
library.php	Biblioteca del usuario
cart.php	Carrito de compras
checkout.php	Proceso de pago
wishlist.php	Lista de deseos
orders.php	Historial de pedidos
order_details.php	Detalle de un pedido
admin_panel.php	Dashboard administrador
developer_panel.php	Dashboard desarrollador

Figura 33. Tabla de vistas adaptadas.

10.2 Patrón de conversión

Antes (MySQL):

php

```
$conn = new mysqli(...);
$result = $conn->query("SELECT ... FROM ...");
while ($row = $result->fetch_assoc()) { ... }
```

Después (MongoDB):

php

```
$db = Database::getInstance();
$cursor = $db->getCollection('...')->find([...]);
foreach ($cursor as $doc) {
    $item = (array)$doc;
    ...
}
```

11. Deploy en Railway

Infraestructura

- Plataforma: Railway (GitHub Student Pack)
- URL pública: https://technogames-production.up.railway.app
- Builder: Dockerfile personalizado
- HTTPS: Automático (Let's Encrypt)

Archivos de deploy

Dockerfile = Imagen con PHP 8.4 + Apache + extensión mongodb

docker-entrypoint.sh = Ajuste dinámico del puerto $PORT

railway.toml = Forzar uso del Dockerfile

Procfile = Eliminado (usa Dockerfile)

.env.example = Plantilla de variables de entorno

Variables de entorno en Railway

MONGODB_URI=mongodb+srv://tg_app:...

MONGODB_DATABASE=technogames

Desafíos resueltos durante el deploy

PHP 8.1 no disponible en Railpack = Cambiar a PHP 8.4

composer install fallaba = Añadir unzip, git, extensión zip

Apache con múltiples MPM = Forzar solo mpm_prefork

Apache no escuchaba en $PORT = Script docker-entrypoint.sh que ajusta el puerto

Variables de entorno no llegaban = Configurar en Railway dashboard

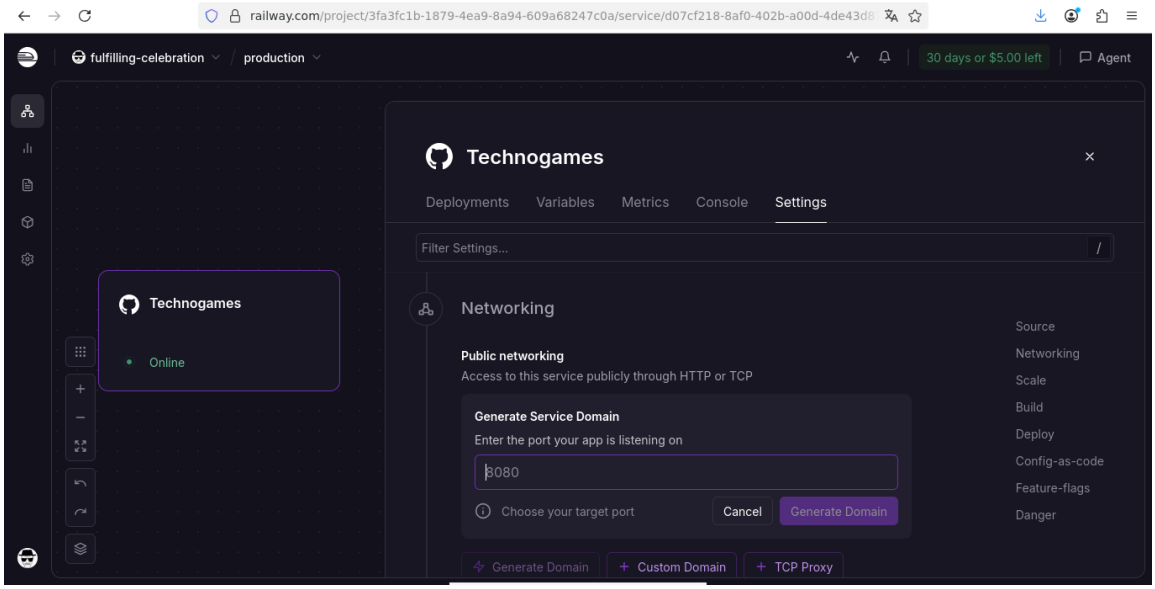


Figura 34. Creando dominio.

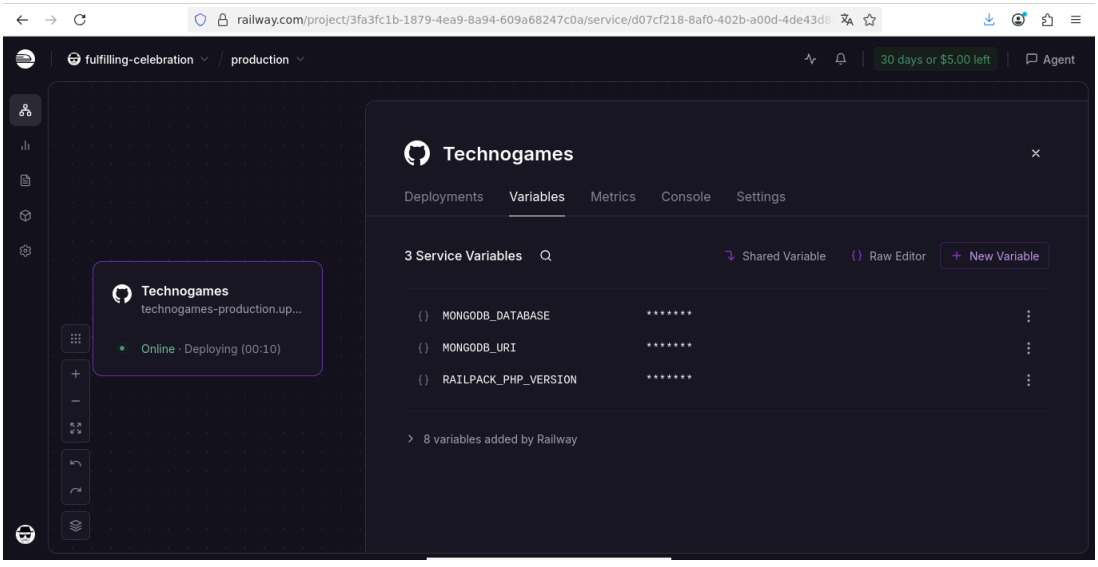


Figura 35. Variables railway deploy.

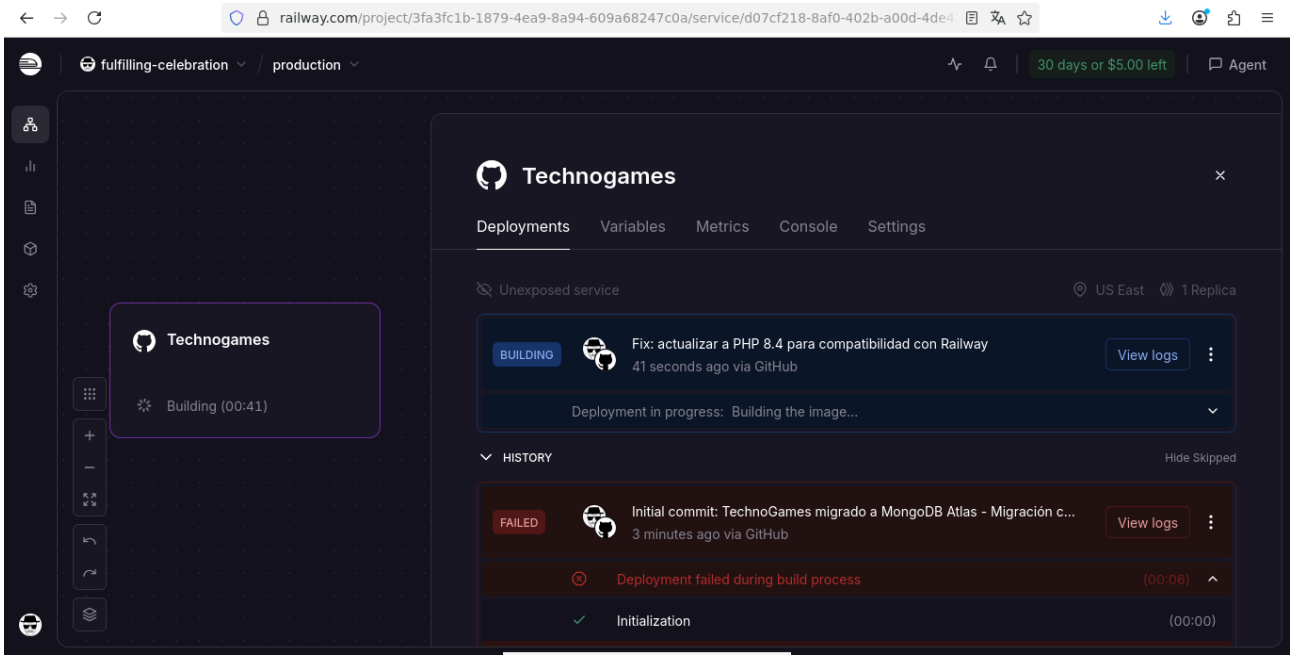


Figura 36. Deploy railway.

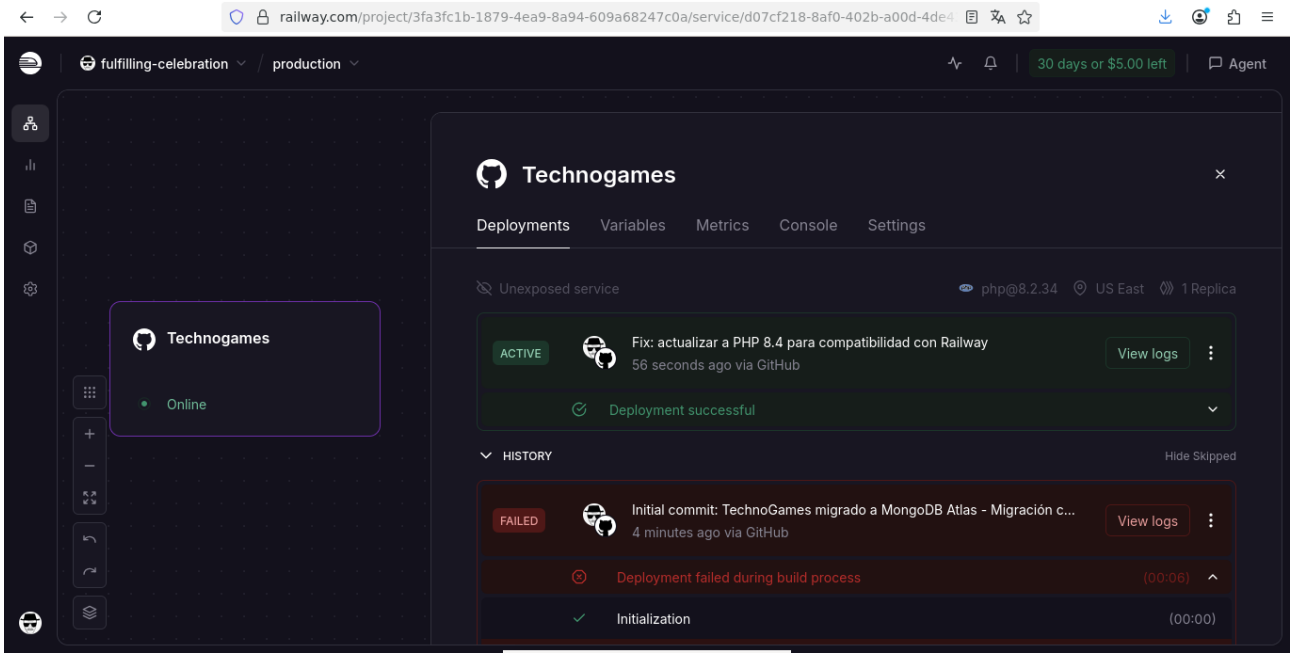


Figura 37. Deploy exitoso.

12. Métricas Finales

12.1 Estadísticas del proyecto

Tablas MySQL	**9**
Colecciones MongoDB	7
Documentos migrados	8 (3 usuarios + 5 juegos)
Archivos PHP adaptados	16 (5 backend + 11 vistas)
Scripts creados	2 (migrate.php, validate.php)
Tiempo de migración	1.52 segundos
Errores en migración	0
Disponibilidad	24/7 (Railway)

Figura 38. Tabla de estadísticas del proyecto.

12.2 Mejoras obtenidas

Aspecto	Antes (MySQL)	Después (MongoDB)
Consultas complejas	Múltiples JOINs	Aggregations optimizadas
Escalabilidad	Vertical	Horizontal (sharding)
Esquema	Rígido	Flexible
Cifrado en tránsito	No	TLS 1.2+
Firewall	No	IP whitelist
HTTPS	No	Automático
Backups	Manual	Automático (Atlas)

Figura 39. Tabla de mejoras obtenidas.

12.3 URL de producción

https://technogames-production.up.railway.app

13. Pruebas de Funcionalidad de la base de datos

Debido a problemas con el front se adjuntan capturas del correcto funcionamiento de la base de datos en la terminal.

```
Atlas atlas-lq3r7d-shard-0 [primary] technogames> // Buscar al usuario 'user1' y mostrar sus datos (sin el hash por seguridad)
| db.usuarios.findOne(
|   { email: "user1@technogames.com" },
|   { nombre_usuario: 1, email: 1, rol: 1, password_hash: 1 } // Incluimos el hash para ver que existe
| );
{
  _id: ObjectId('6ac998bcd96e980fcc0afd04'),
  nombre_usuario: 'user1',
  email: 'user1@technogames.com',
  password_hash: '$2y$12$umSlJ5RzV7ptvoMpgVmTVu/2ZmUpWSx7MUlnmox.AFyKYx8V/BNg2',
  rol: 'user'
}
Atlas atlas-lq3r7d-shard-0 [primary] technogames>
```

Figura 40. Prueba login autentication.

```
Atlas atlas-lq3r7d-shard-0 [primary] technogames> // 1. Usuarios agrupados por rol
| db.usuarios.aggregate([
|   { $group: { _id: "$rol", count: { $sum: 1 } } },
|   { $sort: { count: -1 } }
| ]).toArray();
|
| // 2. Juegos por estado (pendiente/aprobado/rechazado)
| db.juegos.aggregate([
|   { $group: { _id: "$estado", count: { $sum: 1 } } }
| ]).toArray();
|
| // 3. Ingresos totales y comisiones (suma de todas las órdenes)
| db.ordenes.aggregate([
|   { $unwind: "$items" },
|   { $group: {
|       _id: null,
|       total_ventas: { $sum: 1 },
|       ingresos: { $sum: "$items.precio_final" },
|       comisiones: { $sum: "$items.comision" }
|     }
|   }
| ]).toArray();
[ { _id: null, total_ventas: 5, ingresos: 150.7, comisiones: 45.22 } ]
Atlas atlas-lq3r7d-shard-0 [primary] technogames>
```

Figura 41. Panel admin.

```
Atlas atlas-lq3r7d-shard-0 [primary] technogames> // 1. Obtener el ObjectId del desarrollador 'dev1'
| const dev1 = db.usuarios.findOne({ email: "dev1@technogames.com" });
|
| // 2. Contar los juegos del desarrollador
| db.juegos.countDocuments({ desarrollador_id: dev1._id });
|
| // 3. Ver el top de juegos más vendidos del desarrollador
| db.ordenes.aggregate([
|   { $unwind: "$items" },
|   { $match: { "items.desarrollador_id": dev1._id } },
|   { $group: {
|       _id: "$items.juego_id",
|       titulo: { $first: "$items.titulo" },
|       ventas: { $sum: 1 },
|       ganancias: { $sum: { $subtract: ["$items.precio_final", "$items.comision"] } }
|     }
|   },
|   { $sort: { ventas: -1 } },
|   { $limit: 5 }
| ]).toArray();
[
  {
    _id: ObjectId('6ac998bcd96e980fcc0afd09'),
```

Figura 42. Panel desarrollador 1.



Figura 43. Panel desarrollador 2.

```
Atlas atlas-lq3r7d-shard-0 [primary] technogames> // 1. Obtener el ObjectId del juego 'Cyberpunk 2077'
const cyberpunk = db.juegos.findOne({ titulo: "Cyberpunk 2077" });

// 2. Añadir el juego a la biblioteca de user1
db.bibliotecas.updateOne(
  { usuario_id: userId },
  {
    $push: {
      juegos: {
        juego_id: cyberpunk._id,
        titulo: cyberpunk.titulo,
        fecha_adquisicion: new Date()
      }
    },
    $inc: { total_juegos: 1 }
  },
  { upsert: true }
);

// 3. Consultar la biblioteca del usuario
db.bibliotecas.findOne({ usuario_id: userId });
{
  _id: ObjectId('6acc0c9e761941a5e484d1a1'),
```

Figura 44. Prueba biblioteca 1.

```
    titulo: 'Cyberpunk 2077',
    fecha_adquisicion: ISODate('2026-10-11T22:24:30.371Z'),
    order_id: 'ORD-20261011-E0C1CEBC'
  },
  {
    juego_id: ObjectId('6ac998bcd96e980fcc0afd06'),
    titulo: 'Elden Ring',
    fecha_adquisicion: ISODate('2026-10-11T22:24:30.563Z'),
    order_id: 'ORD-20261011-E0C1CEBC'
  },
  {
    juego_id: ObjectId('6ac998bcd96e980fcc0afd07'),
    titulo: 'Hollow Knight',
    fecha_adquisicion: ISODate('2026-10-11T22:24:30.753Z'),
    order_id: 'ORD-20261011-E0C1CEBC'
  },
  {
    juego_id: ObjectId('6ac998bcd96e980fcc0afd08'),
    titulo: 'Hades',
    fecha_adquisicion: ISODate('2026-10-11T23:53:36.639Z'),
    order_id: 'ORD-20261011-B1D4D347'
  },
  {
    juego_id: ObjectId('6ac998bcd96e980fcc0afd09'),
    titulo: 'Stardew Valley',
    fecha_adquisicion: ISODate('2026-10-11T23:53:36.828Z'),
    order_id: 'ORD-20261011-B1D4D347'
  },
  {
    juego_id: ObjectId('6ac998bcd96e980fcc0afd05'),
    titulo: 'Cyberpunk 2077',
    fecha_adquisicion: ISODate('2026-10-17T22:25:10.865Z')
  }
],
total_juegos: 6
}
```

Figura 45. Prueba biblioteca 2.

```
Atlas atlas-lq3r7d-shard-0 [primary] technogames> // Buscar juegos que contengan "RPG" en titulo, descripción o tags
| db.juegos.find({
|   estado: "aprobado",
|   $or: [
|     { titulo: /RPG/i },
|     { descripcion: /RPG/i },
|     { tags: /RPG/i }
|   ]
| }, { titulo: 1, categoria: 1 }).toArray();
[
  {
    _id: ObjectId('6ac998bcd96e980fcc0afd05'),
    titulo: 'Cyberpunk 2077',
    categoria: 'RPG'
  },
  {
    _id: ObjectId('6ac998bcd96e980fcc0afd06'),
    titulo: 'Elden Ring',
    categoria: 'RPG'
  },
  {
    _id: ObjectId('6ac998bcd96e980fcc0afd09'),
    titulo: 'Stardew Valley',
    categoria: 'Simulación'
  }
]
Atlas atlas-lq3r7d-shard-0 [primary] technogames>
```

Figura 46. Prueba búsqueda juegos.

```
Atlas atlas-lq3r7d-shard-0 [primary] technogames> // 1. Obtener el ObjectId del usuario 'user1'
const user1 = db.usuarios.findOne({ email: "user1@technogames.com" });
const userId = user1._id;

// 2. Obtener el ObjectId del juego 'Hades'
const hades = db.juegos.findOne({ titulo: "Hades" });

// 3. Insertar/Actualizar el carrito del usuario
db.carritos.updateOne(
  { usuario_id: userId },
  {
    $push: {
      items: {
        juego_id: hades._id,
        titulo: hades.titulo,
        precio_final: hades.precio_final,
        cantidad: 1,
        agregado_en: new Date()
      }
    },
    $set: { updated_at: new Date() }
  },
  { upsert: true }
);

// 4. Consultar el carrito del usuario
db.carritos.findOne({ usuario_id: userId });
```

Figura 47. Prueba carrito 1.

```
  _id: ObjectId('6acc084517945c5cab05b704'),
  usuario_id: ObjectId('6ac998bcd96e980fcc0afd04'),
  items: [
    {
      juego_id: ObjectId('6ac998bcd96e980fcc0afd08'),
      titulo: 'Hades',
      precio_final: 24.99,
      cantidad: 1,
      agregado_en: ISODate('2026-10-17T22:23:52.720Z')
    }
  ],
  updated_at: ISODate('2026-10-17T22:23:52.720Z')
}
Atlas atlas-lq3r7d-shard-0 [primary] technogames>
```

Figura 48. Prueba carrito 2.

```
Atlas atlas-lq3r7d-shard-0 [primary] technogames> // Contar cuántos juegos aprobados h
| db.juegos.countDocuments({ estado: "aprobado" });
|
| // Buscar juegos de la categoría "RPG"
| db.juegos.find(
|   { estado: "aprobado", categoria: "RPG" },
|   { titulo: 1, precio: 1, precio_final: 1, categoria: 1 }
| ).toArray();
|
| // Buscar juegos que tengan un descuento mayor a 0
| db.juegos.find(
|   { estado: "aprobado", descuento: { $gt: 0 } },
|   { titulo: 1, descuento: 1, precio: 1, precio_final: 1 }
| ).toArray();
[
  {
    _id: ObjectId('6ac998bcd96e980fcc0afd05'),
    titulo: 'Cyberpunk 2077',
    precio: 59.99,
    descuento: 20,
    precio_final: 47.99
  },
  {
    _id: ObjectId('6ac998bcd96e980fcc0afd07'),
    titulo: 'Hollow Knight',
    precio: 14.99,
    descuento: 15,
    precio_final: 12.74
  }
]
Atlas atlas-lq3r7d-shard-0 [primary] technogames>
```

Figura 49. Prueba catalogo.

```
Atlas atlas-lq3r7d-shard-0 [primary] technogames> // 1. Obtener el ID de 'Cyberpunk 2077'
| const cyberpunkId = db.juegos.findOne({ titulo: "Cyberpunk 2077" })._id;
|
| // 2. Insertar una reseña para ese juego por parte de user1
| db.resenas.updateOne(
|   { usuario_id: userId, juego_id: cyberpunkId },
|   {
|     $set: {
|       usuario_nombre: user1.nombre_usuario,
|       calificacion: 5,
|       comentario: "¡Increíble juego! Gráficos y trama de 10.",
|       fecha_creacion: new Date()
|     },
|     $setOnInsert: { votos_util: 0 }
|   },
|   { upsert: true }
| );
|
| // 3. Consultar las reseñas de ese juego
| db.resenas.find({ juego_id: cyberpunkId }).toArray();
[
  {
    _id: ObjectId('6ad3f649761941a5e41b9e85'),
    usuario_id: ObjectId('6ac998bcd96e980fcc0afd04'),
    juego_id: ObjectId('6ac998bcd96e980fcc0afd05'),
    calificacion: 5,
    comentario: '¡Increíble juego! Gráficos y trama de 10.',
    fecha_creacion: ISODate('2026-10-17T22:27:21.464Z'),
    usuario_nombre: 'user1',
    votos_util: 0
  }
]
Atlas atlas-lq3r7d-shard-0 [primary] technogames>
```

Figura 50. Prueba reseñas.

```
Atlas atlas-lq3r7d-shard-0 [primary] technogames> // 1. Obtener el ID de 'Elden Ring'
| const elden = db.juegos.findOne({ titulo: "Elden Ring" });
|
| // 2. Añadir a la wishlist de user1
| db.wishlists.updateOne(
|   { usuario_id: userId },
|   {
|     $push: {
|       juegos: {
|         juego_id: elden._id,
|         titulo: elden.titulo,
|         precio_actual: elden.precio_final,
|         descuento: elden.descuento,
|         agregado_en: new Date()
|       }
|     },
|     $inc: { total_juegos: 1 }
|   },
|   { upsert: true }
| );
|
| // 3. Consultar la wishlist
| db.wishlists.findOne({ usuario_id: userId });
{
  _id: ObjectId('6acc084317945c5cab05b703'),
```

Figura 51. Prueba wishlist 1.

```
  usuario_id: ObjectId('6ac998bcd96e980fcc0afd04'),
  juegos: [
    {
      juego_id: ObjectId('6ac998bcd96e980fcc0afd06'),
      titulo: 'Elden Ring',
      precio_actual: 49.99,
      descuento: 0,
      agregado_en: ISODate('2026-10-17T22:27:59.556Z')
    }
  ],
  total_juegos: 1,
  updated_at: ISODate('2026-10-11T23:05:11.984Z')
}
Atlas atlas-lq3r7d-shard-0 [primary] technogames>
```

Figura 52. Prueba wishlist 2.

Conclusión

La realización de este proyecto permitió comprender y aplicar las principales etapas involucradas en la migración de una base de datos relacional hacia un sistema NoSQL, utilizando MongoDB Atlas como plataforma de almacenamiento para TechnoGames. A través del análisis del esquema original, el diseño de las colecciones documentales y la transformación de los datos, fue posible adaptar la estructura de la información a un modelo más flexible, considerando las necesidades y funcionalidades de la aplicación.

Uno de los aspectos más importantes fue la implementación del proceso ETL mediante scripts en PHP, los cuales permitieron realizar la migración y posteriormente validar los resultados. De acuerdo con las métricas obtenidas, se migraron ocho documentos correspondientes a tres usuarios y cinco videojuegos, en un tiempo aproximado de 1.52 segundos y sin registrar errores durante la ejecución. También se adaptaron los componentes del backend y las vistas para trabajar con MongoDB, además de realizar pruebas de funcionalidades como el inicio de sesión, la consulta del catálogo, la biblioteca, el carrito de compras, las reseñas y la lista de deseos.

Por otra parte, la configuración de MongoDB Atlas y el despliegue en Railway permitieron llevar el proyecto más allá del entorno local, incorporando servicios en la nube y mecanismos de protección como el cifrado mediante TLS y la configuración de accesos a la base de datos. Estas actividades también permitieron identificar y resolver diferentes dificultades técnicas relacionadas con la configuración del entorno, las dependencias de PHP y el despliegue de la aplicación.

En conclusión, el proyecto cumplió con el propósito de migrar la estructura de datos de TechnoGames y adaptar sus componentes principales para trabajar con MongoDB. Además de reforzar los conocimientos sobre bases de datos NoSQL, esta experiencia permitió reconocer la importancia de planificar correctamente una migración, seleccionar una estructura documental adecuada, validar la información y configurar de manera responsable los mecanismos de seguridad. Todo ello constituye una base importante para comprender cómo las tecnologías NoSQL pueden utilizarse en el desarrollo y mantenimiento de aplicaciones web modernas.